Assignments

Project 1, Part A: (I Can't Get No) Satisfaction

One major challenge in logic-based AI is the difficulty of solving the satisfiability problem (usually referred to as SAT). For this assignment, we will ask you to implement two different SAT solvers, commonly known as DPLL and WALKSAT, run your solvers on some test programs, and analyze the performance of the algorithms.

SAT Instances

SAT instances are simply statements in propositional logic, written in conjunctive normal form (CNF). CNF sentences consist of *variables*, *literals*, and *clauses*. In the grammar for this assignment, a variable is a single, case-sensitive character (e.g. “Q”) and a literal is a variable or its negation (“Q” or “~Q”). In a CNF sentence, a clause consists of a disjunction of literals (“Z v ~B v X”). A sentence is a conjunction of clauses, for example:

(A v C v ~B) ^ (Z) ^ (~E v X v Y v W)

A *satisfying assignment* to a SAT instance is an assignment of truth values to variables that makes the CNF sentence true. If such assignment exists for a SAT instance, we say that the SAT instance is *satisfiable*. The problem of determining whether or not a particular SAT instance is satisfiable is NP-complete and thus cannot generally be solved efficiently. For a sentence of *n* variables, it will take O(2_^{n}_) operations to discover an assignment in the worst case - equivalent to checking every possible assignment. Many interesting AI problems are NP-complete or harder, which leads us to consider approximate methods of solution, or exact methods that have good average case running times.

Reading: *Russell & Norvig, Chapters 4,6 (Search, Propositional Logic)*

The Code

We will be providing some handy Java® classes that you can use for this assignment. We have tested it with the Java 1.3 JVM, though it should work on any Java 2 JVM (Java 1.2 and higher). We will be distributing compiled classes and source file packaged as a JAR (Java ARchive) file, as well as posting the Javadoc documentation for the classes. You should not need to look at the source to use the classes, but you can expand the jar file if you need it.

The jar file for this assignment is PL.jar, which contains a package for parsing and representing sentences of propositional logic in CNF. The documentation for these classes is also available. You will need to include this JAR file in your CLASSPATH, and you will probably want to import the package with the statement import techniques.PL.*; before the class definitions.

To parse a String or a File, use the parse methods of techniques.PL.CNF. The parser will only parse CNF sentences, so be sure you are entering them correctly.

To generate a random 3-SAT instance, you can either call CNF.randInstance() or use the CNF.main() method via the command-line.

If you are new to Java, it might be useful to look at the Java® Resources page before you get too involved with your coding. You are, of course, free to use another programming language. If you

are not using Java, you will need to create a your own internal representation for a CNF boolean sentence. You will need to parse the CNF sentences given in our format, as well as create a CNF sentence generator for the experiment detailed below.

Tasks

- You are to try two algorithms (explained in the next section) on the SAT problem. Input to the algorithms should be given as a CNF sentence. Output should be the satisfying assignment, if one is found; if none is found, output an empty assignment. An assignment is a mapping from propositional variables to boolean values.
    1. **(Optional) Brute Force**: You may want to try implementing a brute force (exhaustive search through the solution space) algorithm before DPLL and WalkSAT. This is just so you have something that works and can check your answers from the other two algorithms. You do not need to write up anything for this.
    2. **DPLL**: Implement the DPLL algorithm.
    3. **WalkSAT**: Implement the WalkSAT algorithm.
- Run your algorithms on the test cases below.
- Using the random CNF generator given in the PL.jar code, run your algorithms on random CNF sentences of varying length and measure the speed of the two algorithms.

For this assignment, we present the following SAT instances in text files. Your code should be able to solve these in reasonable time.

Debugging Cases:

- A simple satisfiable sentence
- A slightly more complicated satisfiable sentence
- A simple unsatisfiable sentence
- A slightly more complicated unsatisfiable sentence
- A solveable CNF (Your code should solve this in three minutes or less. Otherwise you need to revise your code)
- Solution to the solveable CNF

Test Cases:

- test 1
- test 2
- test 3
- test 4
- test 5

Algorithms

Davis, Putnam, Logemann and Loveland (DPLL)

This algorithm is popularly known as DPLL and is one of the most widely used methods of solving satisfiability problems exactly. By “exactly” we mean that DPLL is guaranteed either to find a satisfying assignment if one exists or terminate and report that none exists. Therefore, it still runs in exponential time in the worse case.

Reading: ***Cook and Mitchell, “Finding Hard Instances of the Satisfiability Problem: A Survey.”***

The crucial sections of this paper are 1-2.1, which explain notation and the DPLL algorithm. Also read Section 3 closely, it discusses the mathematical properties that distinguish hard SAT problems from easy ones.

GSAT & WALKSAT

GSAT and WALKSAT are stochastic algorithms. In order to speed up the search for a true assignment, they pick random locations in the space of possible assignments and make limited, local searches from those locations. As a result, they might be able to find a solution to a problem that would take too long to solve using DPLL's complete, global search. But because they do not cover the entire search space, there is no guarantee that GSAT or WALKSAT will find a solution to every problem, even if one exists. Therefore, these algorithms can never determine if a sentence is unsatisfiable. Algorithms are frequently categorized as *sound* or *complete*. A sound algorithm guarantees that any answer it returns is correct, but does not necessarily return an answer to every problem. A complete algorithm, on the other hand, will always return an answer. DPLL is complete and sound, while GSAT and WALKSAT are sound, but not complete.

Optional Reading: ***Selman and Kautz, "Domain-Independent Extensions to GSAT: Solving Large Structured Satisfiability Problems."***

Optional Reading: ***Selman, Kautz, and Cohen, "Local Search Strategies for Satisfiability Testing."***

The WALKSAT algorithm evolved from the GSAT algorithm. The first paper has an early exploration of some extensions to GSAT, including "GSAT with Random Walk." The second paper has the actual WALKSAT algorithm, and a brief explanation of its development.

The Write-up

- For each algorithm, turn in the following:
    - A description of your algorithm (not code!) with enough details that others could replicate your results. Include how and why you made any choices (e.g., values you chose for parameters) you made. (**Suggested Length**: 1 - 2 pages for each algorithm)
    - Give a satisfying assignment (if one exists) for the Test cases.
    - Produce a graph charting solution times vs. clause/variable ratio (for a given number of variables) for each of those algorithms. Graph each up to the ratio at which the puzzles become impractical. Average the results from at least 5 trials at each size. The PL.jar code contains a generator of random CNF sentences. The generator will create sentences with the desired ratio for a fixed number of variables. As coded, the number of possible variables is 26.
    - Discuss your results, explaining carefully why you think they came out as they did. Talk about the characteristics of each algorithm, and compare the two algorithms to explain any differences in the results. (**Suggested Length**: 2 - 3 pages.)
    - **Extras**: For extra credit: Try making DPLL as fast as possible. try making DPLL and/or WalkSAT solve more problems by varying your algorithms. Show experimentally how your changes have helped you solve larger problems.
- **Note:**
    - In all parts of this homework, "impractical" means "takes more than 5 minutes to solve."

    - “Speed” should be measured in a machine-independent way. When we collect results, we will ask you to report the number of assignments to the variables your algorithm had to check before solving the problem.
- **Bonus Challenge:**Who couldn’t get no satisfaction?

Collaboration Policy

You may work in teams of two, handing in a single write-up.

Handing in the Assignment

- Hand in answers to the questions and all graphs.
- **Do NOT hand in any code listings.**
- **Staple all materials together.**

Assignments

Project 1, Part B:

Materials

- The project assignment (PDF)
- Code and binaries (JAR) *(updated 10/13)*
- Documentation for the code
- Information about a very efficient SAT solver called zChaff. If you like, you can also work with zChaff.

Notes

(10/3) A new JAR file has been posted. The only difference is that it includes a class

techniques.solvers.DPLL

that you can use as your SAT solver (though you are encouraged to use your own from part a). This jarfile supercedes the previous one and you should *remove*

PL.jar

from your

CLASSPATH

.

(10/2) Converting from a first-order logic sentence to our FOL grammar (clarification of Figure 3)

- **first-order logic:** Ax, y. foo(x) -> bar(y) v Ez. ~equals(x,z)
- **our FOL grammar format:**
  all x all y foo(x) -> bar(y) v exists z ~Equals(x,z)

Important Clarifications:

1. Any time you provide a printout of an assignment, also include a description of it in English (or a with diagram, or something).

2. If a question asks you to provide an assignment, you do not need to provide _all_ assignments. One will do.
3. For items 11 and 12, we mean the following:
    - you should create an axiom that defines what it means to be a well-connected set of rails (i.e., that it's possible to move from each rail to each other rail)
    - then, having asserted well-connectedness, list the actual assignment returned for the "connects" relation: this relation represents the track layout found by your SAT solver.

4. There is a typo in item 13. the definition of liveness from "formula 8" refers to your revised definition of 'deadlock' from question _10_.
5. Yes, the extra credit is worth actual points on 1b, just as it was on 1a. (note that an "extra credit" problem is distinct from a "bonus challenge!")
6. Yes, please form a group of two (if you like) for this assignment, as well. you can work with the same partner for 3 of the 5 projects.

Answers to Some Common Questions:

1. "Deadlocked" refers to the whole situation: a situation is deadlocked if both trains *can not* move. If at least one train has a move available, then we do not consider that situation to be deadlocked.

2. Your solution to the deadlock questions cannot involve:
    - changing the non-determinism of the world dynamics. a question was asked if it was ok for each train to have a plan of action, and to broadcast that plan. that makes the dynamics deterministic, and it would mean the other trains would know what to avoid. this is not really the spirit of the question, since it's not allowable to have…
    - knowledge about the next situation (i.e. "peeking" into the future). your solutions must involve only what is known about the _current_ situation. try to think about whether an additional relation on the trains would "break symmetry" in a way that avoids deadlock.
    - removing the "Safe"ness of the situations S1 and S2.

3.
4. Running out of memory:
   you may have gotten "out of memory" errors during the process of changing the FOL sentence into a CNF sentence. this is not a bug in the code, and it's probably not the 'fault' of any individual axiom in your FOL sentence. it's just the nature of exponential growth: converting a sentence from FOL to PL and then to CNF grows exponentially bad with the number of propositions. exponential growth is bad news; if you didn't believe that before, you should now!
   that said, it _is_ possible to do the assignment within the standard memory constraints of the JVM (128 MB). if you are running out of memory, try looking at the following things in your set of axioms:
    - do you have any axioms that are redundant, or duplicating the work of other axioms? some people made their definition of "legal" to do the work of both "legal" and of enforcing the world dynamics. a little parsimony goes a long way.
    - do you have statements that say something like "all s" when simply instantiating the statement for S1 would do?
    - do you have any axioms left over from previous questions in the assignment that are no longer needed?
      there is really no guaranteed method for avoiding memory problems; if you have

trouble with it, you'll just need to play around with your sentences until they get small enough.

What to Turn In:

Please note that the axioms you come up with should assume as little as possible about the arrangement of rails or number of trains; they should be generally applicable to any world with any number of trains and rails. One of the grading criteria will be the generality of your axioms.

1. - the formulas 1-6, in the language of figure 3 (hereafter denoted as 'homework language').

2. - the two new axioms, in the homework language.

3. - a printout of the found satisfying assignment.
   - in english, a description of the assignment, including the locations of the trains in S1 and S2.

4. - how you used DPLL to show no crashes (a description of your method)
   - why or why not to use WalkSAT

5. - the new definition of "legal" in homework language
   - an English or standard FOL description of your new definition
   - any changes to the connects relation?
   - whether the domain is still safe
   - how you used DPLL to show safeness (a description of your method)

6. - homework language description of layout 2
   - how would you show it is not safe? (describe your method)
   - printout demonstrating safeness (either an assignment, or that your procedure returned 'null' for no assignment)

7. - the new definition of "legal" in the homework language
   - an English or standard FOL description of your new definition
   - whether the domain is still safe
   - a description of your method for showing that it was still safe
   - a safe configuration you found (or more, if you found more than one) for S1 and S2

8. - the axiom, in homework language and English/FOL, that asserts that S1 is deadlocked.
   - a description of your method for showing no deadlock in layout 1

9. - a description of your method for showing deadlock in layout2
   - the members of the ON relation, if your proof method produced an assignment
   - a printout of the assignment, if you have one

10. - the new axiom for legal (in homework language, and English or FOL)
    - any new axioms or relations that you added
    - a printout of the assignment for the given arrangement

11. no actual programming is required here. just think about it, and write about what you _would_ do to axiomatize, in FOL, the notion of "eventually moving". then, write about whether the approach of this assignment (i.e. of turning an FOL sentence into a propositional one) would

work for proving/disproving the correctness of that condition.

12. the words “second implication” are a typo: there is only one implication in formula 5. (the formula describing the dynamics.)

Course Info

Instructors

Prof. Tomás Lozano-Pérez
Prof. Leslie Kaelbling

Departments

Electrical Engineering and Computer Science

As Taught In

Fall 2002

Level

Graduate

Topics

Engineering
Computer Science
Algorithms and Data Structures
Artificial Intelligence
Science

Learning Resource Types

notesLecture Notes
assignmentProgramming Assignments

Assignments

Project 1, Part C:

Materials

- The project assignment
- The Resolution-Refutation Proof Checker (JAR)
- The project files:
  - sibling.txt (TXT)
  - blocks.txt (TXT) (fixed inconsistencies with ‘on’ predicate and ‘clear’ axiom)
  - blocks_extracredit.txt (TXT) (‘move’ axiom is different than blocks.txt version)
  - trains1.txt (TXT)
  - trains2.txt (TXT)

Announcements

- About the jarfile:
  - The jarfile contains both compiled classes and source files. To use it, you can either put it in your class path and type `java RRPC.Checker`. Or, on MacOS X and

Windows with JDK you can just double-click the jarfile and the checker will launch.
    - If you are saving/loading the proofs as you work on them, you should be sure to **Output Proof** the proof as well as saving them. **Output Proof** will write the proof in a human-readable format.

- About blocks.txt:
    - The file has been updated to correct the swapped arguments in the
      clear
      axiom. (This bug should not have actually affected your ability to do the proof, but it's fixed now.)
    - There were two versions of the "on" predicate: one with two arguments, and one with three. (This meant that you could not use resolution on the two different versions. However, it really didn't cause any problems because you did not need to do resolution across the two different versions to do the proof.) This has now been corrected.
    - The extra credit version, blocks_extracredit.txt, has only one version of "on": the three argument version. It also contains a different version of the "move" axiom, with an additional predicate.

What to Turn In

- Your proofs, using the **Output Proof** command

- For each proof, please also:
    - Provide a brief (not more than a paragraph) proof sketch
    - Point out any key resolution steps in your proof

- For the extra credit, in addition the proof, explain why the solution depends on having an axiom that explicitly mentions
  clear(x)
  after moving
  x
  (even though the solution to question 3 didn't depend on such a thing).
    - Correction: don't worry about justifying the 3-argument version of the "on" predicate.

Assignments

Project 2a: Bayes Net Learning

In this project, you will implement Bayes net learning. That is, you will write a program that will read in some data and come up with a Bayesian network that best matches the data. Reading and understanding Nilsson's Chapter 20, section 1 is necessary for doing this assignment.

This assignment is broken up into parts that will take you step-by-step through building a program for Bayes net learning.

Part 0: Designing the Data Structure for a Bayes Net

Please read through this whole assignment and make sure you understand everything you're asked to do. Then, design and implement a data structure for a Bayes net, taking into account what you'll have to do with the data structure.

- **(Turn-in) 10 points** Describe your data structure using plain English and draw your data structure. In particular, describe where and how the links and conditional probability tables are stored in the structure.

Part 1: Estimating the CPTs

In this part, you are given the structure of a network and need to compute the conditional probability tables (CPTs). For this assignment, we will assume that there is no missing data, so this is just a matter of counting. For parts 1 and 2, simply use the raw counts (just as Nilsson does).

Your Tasks:

- Write a procedure for computing the estimated CPTs given a network structure and data.
- Test the correctness of your procedure using the network and data in Data A. The "correct" CPTs are in the file.
- **(Turn-in) 10 points** Compute the CPTs of the network and data in Data B.

Part 2: Computing the Score of a Network

Pretend that you have the network already defined for you. That is, you already know the network structure and the conditional probability tables. How would you know whether it is a good network? This first part is about computing the score of a given network so that you can compare different networks.

We can compute the score of a network in any way we want, but a *good scoring metric* would take into account two things: (1) probability of the data given the network; (2) complexity of the network. For part 2, please use the scoring metric in Nilsson's chapter.

Your Tasks:

- Write a procedure for computing the score of a network using the scoring metric in Nilsson's chapter.
- Test the correctness of your procedure using the network and data Data A. The "correct" score is in the file.
- **(Turn-in) 10 points** Compute the score of the network and data in Data B. Please report both **|B|** and **m** seperately.

Part 3: Generating an Initial Network

Now you have the tools for searching through the space of network structures. However, like many other search problems, exhaustive search through the space of all possible network structures is not practical. In Part 4, you will need to implement some kind of greedy search, and as you should know by now, if you are not doing exhaustive search, your result will depend on how you choose the starting point. So in this part, you will think about how to choose an initial network. Possible candidates for an initial network are:

- Network with no dependencies
    - This is a network with no arrows between any nodes.

- Fully connected network
    - This is a network in which you randomly choose a top node (root of the tree), which has no parents, then randomly choose the next node whose parent is the root node, then choose the next node whose parents are the first two nodes, etc. In general, when a node is chosen as the next one to be added to the network, its parents are all of the nodes already chosen. If this description is not clear, please talk to one of the TAs.
- Randomly connected network
    - This is a network in which the nodes are randomly connected. You just need to make sure that there are no directed cycles.
- Maximum spanning tree based on mutual information (if you do not choose to implement this one, you don't need to explain why)
    - This is for the curious minds. If you're interested in this, come and talk to the TAs.

Note that any network you choose (here and also throughout your search) cannot have directed cycles. Also note, from this point on, you may use bayesian correction if your network has some probability 0 events.

Your Tasks:

- Write two procedures that will generate an initial network (any two of the ones mentioned above)
- **(Turn-in) 5 points** Describe how you've made sure that your network does not have any directed cycles
- **(Turn-in) 5 points** Generate two initial network for each of the two data sets: Data A and Data B. Draw the four networks and describe how you generated them.

Part 4: Searching

Given the initial network in Part 3, implement a local greedy search to find a *locally optimal network.* (as described in Nilsson's Chapter).

Your Tasks:

- Write a program to do the search. You need to define a stopping criterion so that your program does not keep searching.
- **(Turn-in) 10 points** Explain your search algorithm well enough so that someone else can implement it (cf. Project 1a) **Note:** Please use some manner of randomization into your algorithm, either in your initialization step or in the search itself.
- **(Turn-in) 15 points** For one initialization method, run your program 3 times each on data in Data A and Data B to find a locally optimal network. Draw the resulting the networks (there should be 6) along with the CPTs. Do you get different results each time you run it? Why or why not?

Part 5: Experiment

In this part, you will do two experiments: one with Part 2, and another with either Part 3 or Part 4.

Your Tasks:

- Experiment with Part 2: Try using a different scoring metric, for example, by adjusting the weighting of some of the terms. Run it on the two data sets several times and compare the two different scoring metrics you've used.
    - **(Turn-in) 15 points** How did you change the scoring metric? What was your hypothesis about how this would affect the final networks? Draw the resulting final networks and explain the difference (if any).
- Experiment with Part 3 or Part 4. Make a non-trivial change to your algorithm and see if you get different final networks. The changes can be in the initialization (e.g. trying another method) or in your search itself. If you decide to modify a parameter only, then do this in a systematic way; that is, try various settings of the parameter and plot the changes in some performance measure. Compare the scores of the final networks using different initial networks. Or try a different search algorithm. See if that results in a different final network. Compare the scores.
    - **(Turn-in) 15 points** Explain your experiment. What did you do differently? What was your hypothesis about how this would affect the final networks and the score of the networks? For each condition, run it on the data three times and draw a graph comparing the two conditions. Discuss your graph.